

Deep Learning for Sequence Prediction --- Recurrent Neural Networks

Sunita Sarawagi
IIT Bombay
sunita@iitb.ac.in

Sequence Modeling tasks

- Classify sequences $x_1, \dots, x_n \rightarrow y$.
 - Sentiment classification, question intent classification, normal versus abnormal traffic
- Next word in a sequence $x_1, \dots, x_n \rightarrow x_{\{n+1\}}$
 - Language modelling, forecasting.
- Label per token in sequence $x_1, \dots, x_n \rightarrow y_1, \dots, y_n$
 - POS, Named entity recognition
- Sequence prediction tasks $x_1, \dots, x_n \rightarrow y_1, \dots, y_m$
 - Translation, conversation assistant, speech recognition

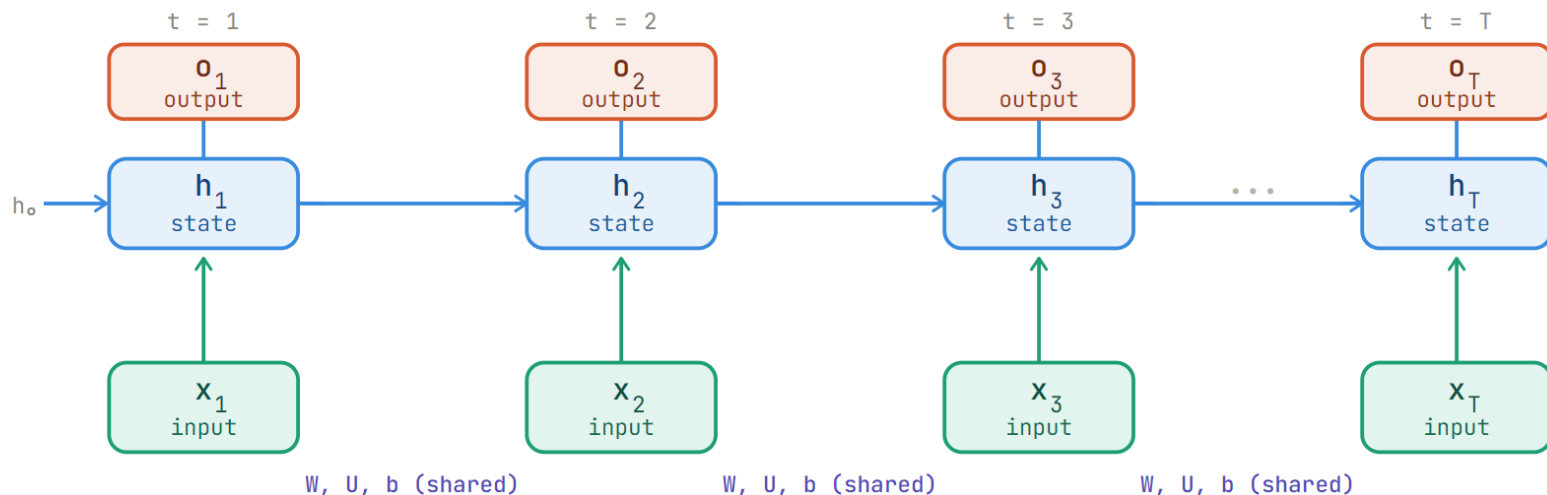
Why existing network architectures will not work

- Feed forward networks
 - Sequences can be of variable length. Can train only fixed number of parameters.
 - Even when sequences were of fixed length, do we really need to train a parameter per input token?
- CNNs
 - Designed to capture local patterns whereas in sequences interaction may be non-local. E.g.
 - He, after a vigorous workout, is enjoying a delicious breakfast with his friends.

RNN: Recurrent Neural Network

- A model to process variable length 1-D input
- In CNN, each hidden output is a function of corresponding input and some immediate neighbors.
- In RNN, each output is a function of a 'state' summarizing all previous inputs and current input. State summary computed recursively.
- RNN allows deeper, longer range interaction among parameters than CNNs for the same cost.

RNNs: Basic type



STATE $h_t = \sigma(W h_{t-1} + U x_t + b)$

OUTPUT $o_t = V h_t + c$

WEIGHTS W : state $\leftarrow$ state U :
state $\leftarrow$ input

OUTPUT V : output $\leftarrow$ state b, c :
biases

□ Input x_t □ Hidden state h_t □ Output o_t □ Shared weights

RNN: forward computation example

Input:

parameters: $W, U, b, c, \bar{V}$

Input: $x_1, x_2, \dots, x_T$: $x_i \in \mathbb{R}^d$

Input: h_0 : $h_0 \in \mathbb{R}^k$

$k =$ hidden state size

Output size: m

Output

$o_1, \dots, o_T$ $o_t \in \mathbb{R}^m$

for $t = 1$ to T {

$$h_t = \sigma(W h_{t-1} + U x_t + b)$$

$$o_t = V h_t + c$$

}

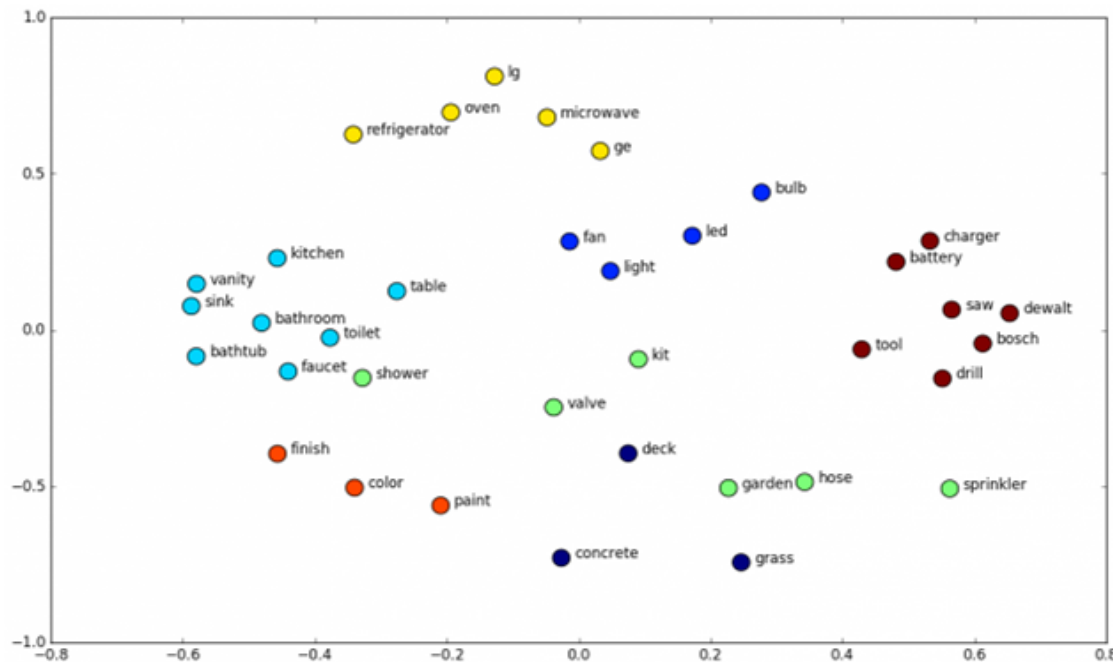
Return $o_1, o_2, \dots, o_T$

RNN: forward computation example

Input: Text, Task: Text classification

- Input sequence: $x_1, x_2, \dots, x_n$
- Each x_t instead of a real-valued vector is a discrete word.
- The task is to predict a label for this text
 - Example: Spam or Not-spam
 - Sentiment or star rating of a review
 - Topic of a document
 - Intent of a query
- How to convert a discrete word into a real-value for input into a RNN?

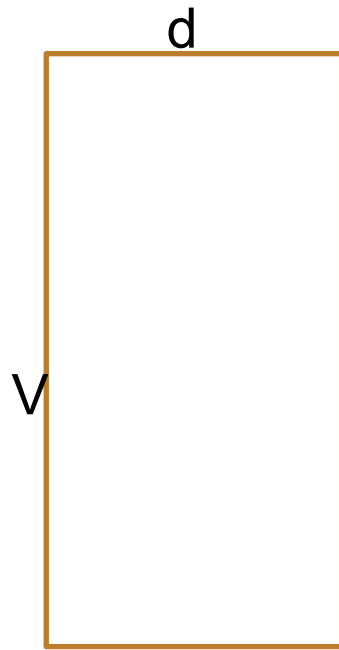
Word-embeddings: Represent each word as a point in a multi-dimensional space..



Similar words are close together in that space.

Word embeddings

- Choose the embedding size: d
 - Typical value for words: 128
- Choose the vocabulary size: V
 - Typical size: 30 thousand.
 - Rest are broken into subwords or UNKs.
- Embedding matrix: E
- If an input word x_t is a one-hot vector of size V , then embedding is obtained as $E^T x_t$



Example

Vocabulary:

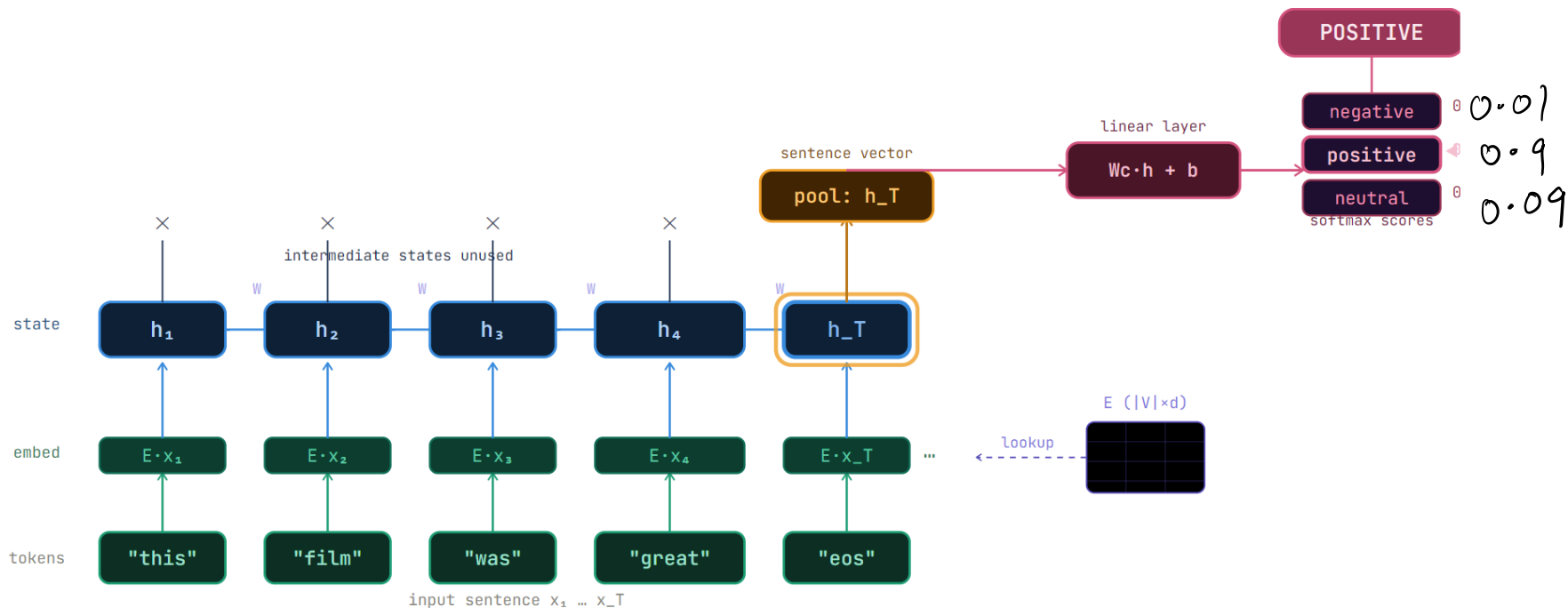
Man, woman, boy,
girl, prince,
princess, queen,
king, monarch



	1	2	3	4	5	6	7	8	9
man	1	0	0	0	0	0	0	0	0
woman	0	1	0	0	0	0	0	0	0
boy	0	0	1	0	0	0	0	0	0
girl	0	0	0	1	0	0	0	0	0
prince	0	0	0	0	1	0	0	0	0
princess	0	0	0	0	0	1	0	0	0
queen	0	0	0	0	0	0	1	0	0
king	0	0	0	0	0	0	0	1	0
monarch	0	0	0	0	0	0	0	0	1

Each word gets
a 1x9 vector
representation

RNN for text classification



EMBED $e_t = E \cdot x_t$ ($E \in \mathbb{R}^{|V| \times d}$)

POOL $v = h_T$ (final hidden state)

STATE $h_t = \sigma(W h_{t-1} + U e_t + b)$

CLASSIFY $\hat{y} = \text{softmax}(W_c v + b_c)$

■ tokens / embeddings ■ hidden states h ■ sentence vector (pool) ■ classifier / output ■ weight matrices

RNN: forward computation example

Input:

parameters: $W, U, b, c, \bar{V}, E$

Input: $x_1, x_2, \dots, x_T$: $x_i \in \mathbb{R}^k$

Input: h_0 : $h_0 \in \mathbb{R}^k$

Output size: C

Output

$P(y|x_1 \dots x_T)$

for $t = 1$ to T {

$$e_t = E^T x_t$$

$$h_t = \sigma(W h_{t-1} + U e_t + b)$$

$$P(y|x_1 \dots x_T) = \text{Softmax}(V h_T + c)$$

$E \in \mathbb{R}^{O \times d}$: $O = \text{vocab size}$

$x_i \in \text{one-hot vector of size } O$

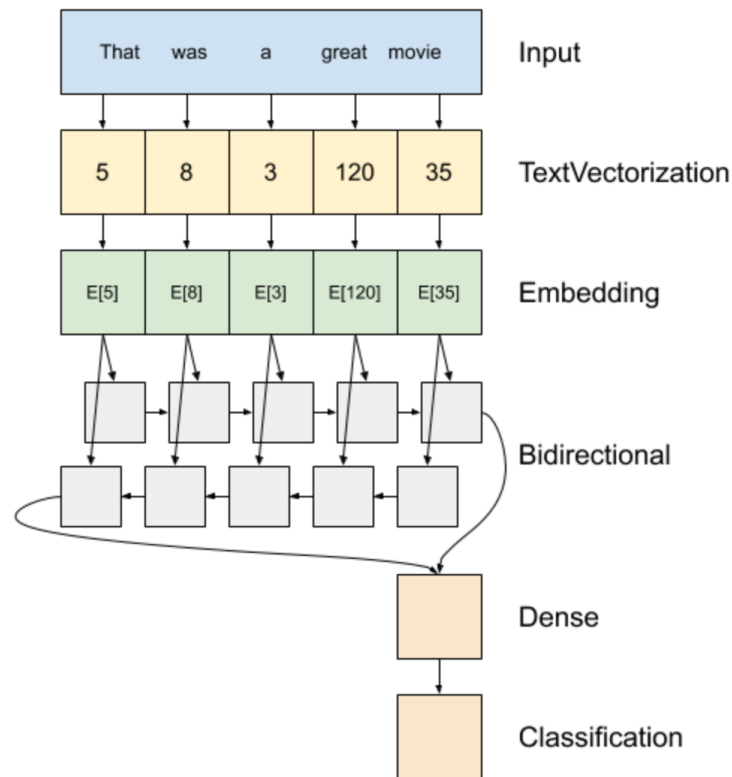
$k = \text{hidden state size}$

$C \leftarrow \# \text{ of class labels.}$

$V \in \mathbb{R}^{C \times k}$

Text classification with a bidirectional RNN.

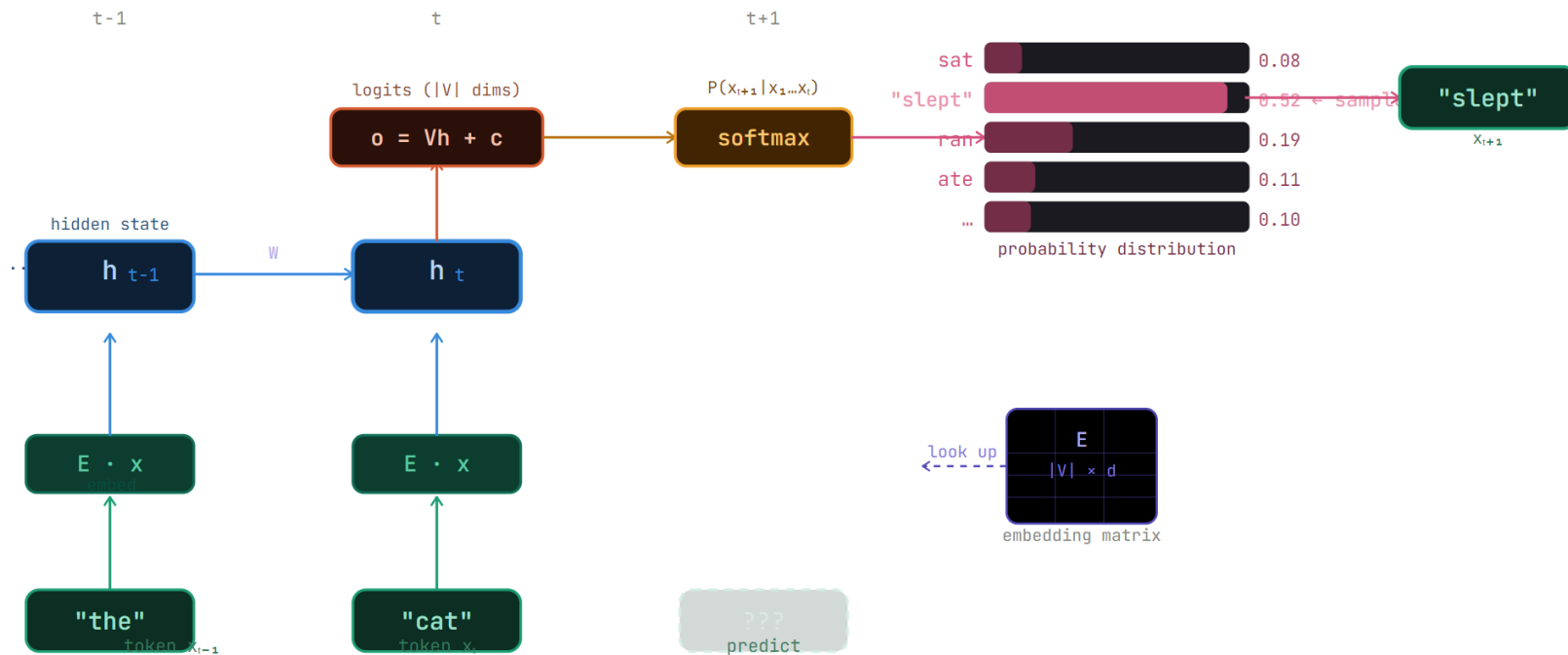
All parameters including the word embeddings are trained end-to-end.



Input: Text, Task: Predicting next word in a sentence

- Input sequence: $x_1, x_2, \dots, x_n$
- Each x_t instead of a real-valued vector is a discrete word.
- The task is to predict the following word $x_{\{n+1\}}$
- Output: Softmax over the RNN hidden state h_t to predict the multinomial distribution over the next token.

Next token prediction using RNNs



EMBED $e_t = E x_t \quad (E \in \mathbb{R}^{|V| \times d})$

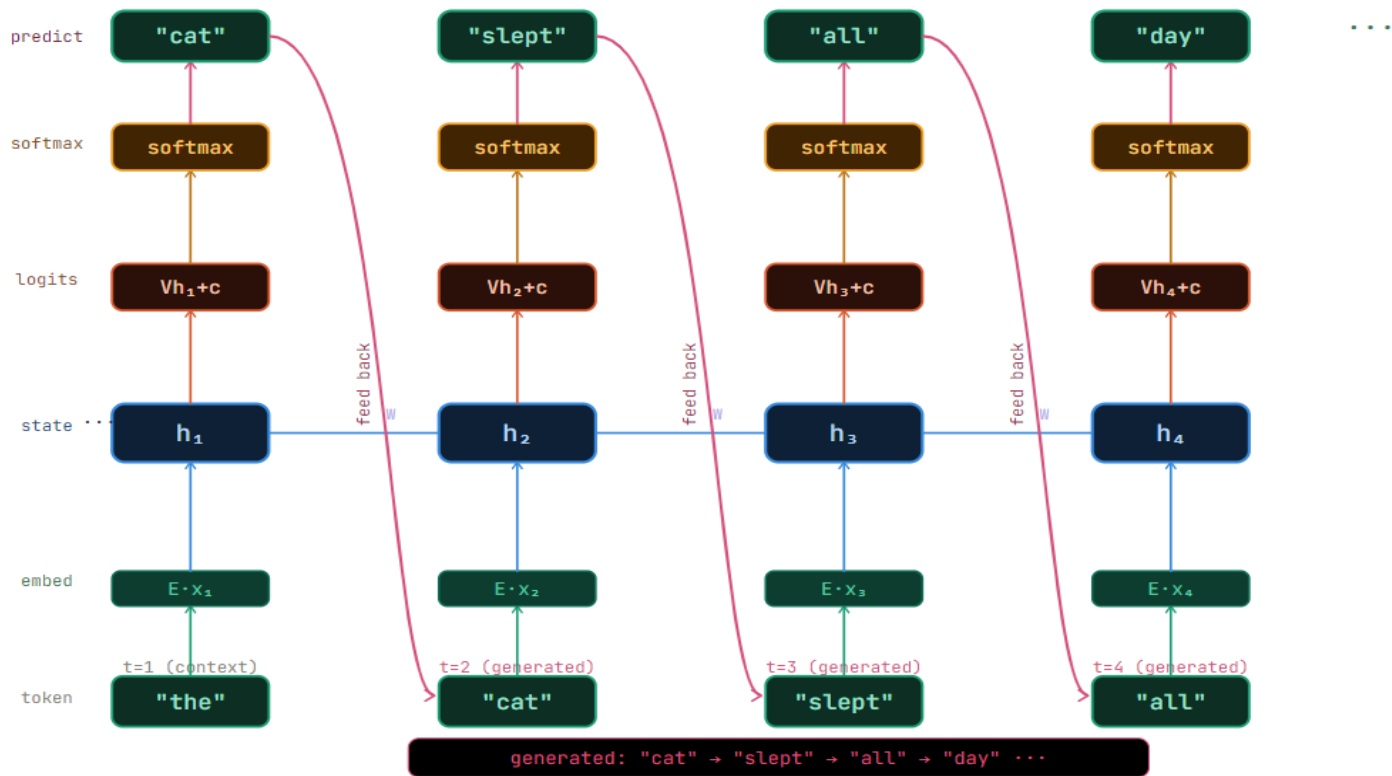
STATE $h_t = \sigma(W h_{t-1} + U e_t + b)$

LOGITS $o_t = V h_t + c$

PREDICT $P(x_{t+1} | \dots) = \text{softmax}(o_t)$ then sample

| tokens / embeddings ■ hidden state h ■ logits o ■ softmax ■ probability / sample ■ weight matrices

Completing a sentence using RNNs: Autoregressive generation



STATE $h_t = \sigma(Wh_{t-1} + UEx_t + b)$

SAMPLE $\hat{x}_{t+1} \sim P(x_{t+1})$ then feed back as x_{t+1}

PREDICT $P(x_{t+1}) = \text{softmax}(Vh_t + c)$

LOOP repeat until <eos> or max length

■ tokens / embeddings ■ hidden state h (recurrent) ■ logits ■ softmax ■ feedback (predicted $\rightarrow$ next input)

RNN: forward computation example

Input:

parameters: $W, U, b, c, \bar{V}, E$

Input: $x_1, x_2, \dots, x_T$: $x_i \in \mathbb{R}^k$

Input: h_0 : $h_0 \in \mathbb{R}^k$

Input: EOS token-id.

- Assume: $k = d$

for $t = 1$ to T {

$$e_t = E^T x_t$$

$$h_t = \sigma(W h_{t-1} + U e_t + b)$$

}
for $t = T+1$ to max-sequence length {
 $p(x_t | x_1, \dots, x_{t-1}) = \text{softmax}(E h_{t-1} + c)$

$E \in \mathbb{R}^{O \times d}$: $O = \text{vocab size}$

$x_i \in \text{one-hot vector of size } O$

$k = \text{hidden state size}$

sample $x_t \sim P(x_t | x_1 \dots x_{t-1})$

$$e_t = E^T x_t$$

$$h_{t+1} = \sigma(W h_t + U e_t + b)$$

if $x_t = \text{"EOS"}$ - exit the for-loop.

}

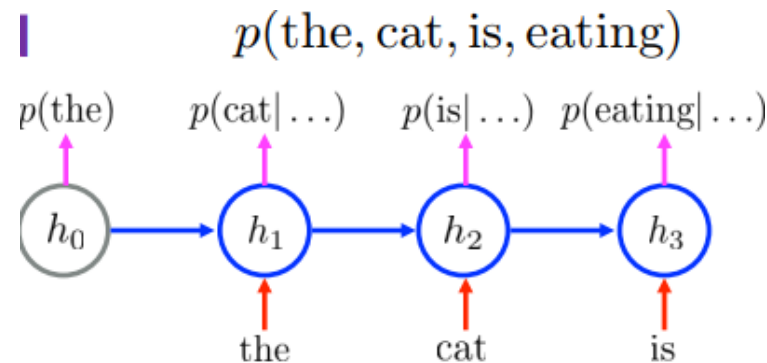
Output $x_{T+1}, x_{T+2} \dots$

In equations...

In equations...

Training a sequence model

- Maximum Likelihood
- $P(x) = \prod_{t=1}^n P(x_t | x_1, \dots, x_{t-1})$
- $h_t = x_1, \dots, x_{t-1}$
- Mechanism of training
 - Input to RNN is the true tokens upto time $t-1$
 - Output is the probability distribution over tokens
 - Maximize the probability of the correct token.



Training data

$$D \equiv \{(x^1, y^1), \dots, (x^N, y^N)\}$$

LM: $y^i = x^i$ shifted by one to the left

Example x^i : The cat is sleeping

x_1^i x_2^i x_3^i x_4^i

y^i

cat is sleeping EOS

y_1^i y_2^i y_3^i y_4^i

The training loop:

Input: a batch of sentences. $(x^1, y^1) \dots (x^B, y^B)$

x^i, y^i = discrete of length T .

In practice if a sentence is of length less than T , we pad the sentence.

1* Forward pass *

h_0 & Initialize state; loss = 0

for $t = 1$ to T {

$h_t^i \leftarrow \text{state}_B(h_{t-1}, E x_t^i)$

$\leftarrow i = 1$ to T .

loss $+ = - \sum_{i=1}^B \log P(x_t^i | h_t)$

$\leftarrow$ obtained using softmax

}

return loss;

Back propagation for RNNs

